

Detección de Rostros Sintéticos Generados por StyleGAN3: Diagnóstico de Sesgos en Conjuntos Públicos y Diseño de un Pipeline Acelerado en Cómputo Paralelo

Jason Barrantes Sánchez Melany Ramírez Anchía Walter Bowyer Carpenter Mauro Espinoza Hernández

Bachillerato en Ingeniería en Ciencia de Datos, Universidad LEAD

San José, Costa Rica

{jason.barrantes, melany.ramirez, walter.bowyer, mauro.espinoza}@ulead.ac.cr

RESUMEN

Se presenta un sistema de detección de rostros sintéticos generados por StyleGAN3, desarrollado sobre el clúster Kabré del Centro Nacional de Alta Tecnología. El aporte principal del trabajo no es el clasificador en sí sino el diagnóstico que lo precedió: un modelo Vision Transformer que alcanzaba 82 % de aciertos sobre el conjunto público de referencia detectaba solo el 23 % de los rostros que el mismo generador producía en tiempo real. Mediante tres hipótesis falsables evaluadas experimentalmente se descartó el sesgo de compresión ($d = 0,093$) y el del filtro de remuestreo (dispersión de 3 puntos porcentuales entre seis filtros), y se confirmó que el conjunto público había sido generado con un parámetro de truncamiento bajo, lo que produjo un atajo estadístico explotable: la detección cae de 100 % con $\psi = 0,40$ a 23 % con $\psi = 1,00$. Corregida la composición del conjunto y la estrategia de entrenamiento, y sustituido el transformador de parches por un conjunto de ConvNeXt-Tiny y Swin-T, el sistema alcanza 99,7 % de aciertos y AUC-ROC de 0,9999 sobre rostros generados con semillas nunca vistas, empleando la quinta parte de los parámetros. El pipeline de preprocesamiento paralelo sostiene 2 198 imágenes/s con dieciséis procesos frente a 511 imágenes/s en ejecución secuencial.

Palabras clave—Vision Transformer, ConvNeXt, detección de deep-fakes, StyleGAN3, sesgo de conjunto de datos, cómputo paralelo, Kabré.

I. INTRODUCCIÓN

Los modelos generativos han alcanzado un grado de realismo que vuelve inviable la verificación humana de autenticidad. StyleGAN3 [2] introdujo una formulación equivariante a traslación y rotación que elimina los artefactos de *aliasing* característicos de sus predecesores, produciendo rostros cuya distinción a simple vista resulta prácticamente imposible.

Este trabajo aborda la clasificación binaria de rostros humanos: dada una imagen RGB $I \in \mathbb{R}^{H \times W \times 3}$, determinar si corresponde a una fotografía auténtica ($y = 1$) o a una síntesis de StyleGAN3 ($y = 0$). En el marco del curso de Computación Paralela y Distribuida, el problema se aborda además desde la perspectiva del rendimiento computacional: se cuantifica el beneficio del paralelismo multiproceso en el preprocesamiento y el de la aceleración por GPU en el entrenamiento.

El hallazgo central del trabajo, sin embargo, fue

metodológico. Un modelo que parecía funcionar correctamente resultó estar explotando una particularidad del conjunto de datos público en lugar de aprender las características del generador. La sección IV documenta el proceso de diagnóstico que reveló esa situación, que consideramos la contribución más transferible de este trabajo.

A. Contribuciones

1. La identificación y caracterización experimental de un sesgo de truncamiento en un conjunto de datos público ampliamente disponible, con la demostración de que un clasificador entrenado sobre él no generaliza al generador que dice representar.
2. Un protocolo de evaluación que separa explícitamente la distribución de entrenamiento de la de prueba mediante semillas disjuntas del espacio latente, permitiendo medir generalización real.
3. La evidencia de que, para esta tarea, una arquitectura convolucional supera a un transformador de parches con un tercio de los parámetros, resultado contrario a la expectativa inicial del proyecto.
4. Un pipeline reproducible en infraestructura HPC, verificado mediante regeneración completa desde cero.

II. TRABAJO RELACIONADO

Wang *et al.* [6] demostraron que un clasificador convolucional entrenado sobre imágenes de un único generador puede generalizar a arquitecturas no vistas, sugiriendo la existencia de artefactos comunes entre familias de modelos generativos. Nuestros resultados matizan esa conclusión: la generalización falla cuando la distribución de entrenamiento cubre solo una porción del espacio de salida del generador, aun tratándose del mismo generador.

Los Vision Transformer [3] han mostrado desempeño competitivo frente a las arquitecturas convolucionales en clasificación de imágenes, y diversos trabajos han explorado su uso en detección de contenido sintético argumentando que la autoatención captura relaciones espaciales de largo alcance. La sección D presenta evidencia en sentido contrario para esta tarea concreta.

TABLA 1
CONFIGURACIÓN DE LA INFRAESTRUCTURA

Recurso	Especificación
GPU	NVIDIA L40S, 46 GB VRAM
CPU	Intel Xeon Silver 4416+, 20 núcleos
Núcleos asignados	16 por trabajo
Memoria	64 GB
CUDA / cuDNN	12.1 / 9.1
Almacenamiento	Lustre, cuota de 20 GB
Planificador	SLURM, límite de 4 h por trabajo

En cuanto a infraestructura, el uso de Polars [8] para procesamiento columnar multihilo y de Dask [7] para cómputo distribuido perezoso está documentado en la literatura de ingeniería de datos, aunque su aplicación a pipelines de visión orientados a detección de contenido sintético permanece poco explorada.

III. METODOLOGÍA

A. Conjunto de datos

Se emplea el conjunto público *10000 Real vs Fake Faces (StyleGAN3)*, disponible en Kaggle bajo licencia CC-BY-NC-SA-4.0. Contiene 20 000 imágenes balanceadas (10 000 auténticas y 10 000 sintéticas) en formato PNG sin pérdida a resolución 256×256 .

Conviene señalar una discrepancia detectada durante el trabajo: la documentación del conjunto refiere una resolución nativa de 1024×1024 , pero la inspección de los archivos reveló que fueron distribuidos ya reducidos a 256×256 . Este detalle resultó relevante para descartar el entrenamiento a mayor resolución (sección C).

B. Infraestructura de cómputo

Todo el procesamiento se ejecutó en el clúster Kabré del CeNAT, partición *nukwa-140s*. La tabla 1 resume la configuración empleada.

C. Diseño del pipeline

El pipeline se organiza en seis etapas. El inventario de metadatos se construye con Polars, que aprovecha los núcleos disponibles sin configuración explícita. El preprocesamiento — decodificación, redimensionado y normalización — se reparte entre dieciséis procesos mediante `ProcessPoolExecutor`, evitando la limitación del *Global Interpreter Lock* de CPython. Los tensores resultantes se almacenan en fragmentos `.npy`.

Una decisión de diseño relevante fue almacenar las imágenes como enteros de ocho bits sin normalizar, aplicando la normalización de ImageNet en tiempo de carga. Frente al almacenamiento en punto flotante de 32 bits, esto reduce el espacio en disco por un factor de cuatro (12 GB a 2,9 GB), lo cual resultó determinante dada la cuota disponible. El costo computacional

de normalizar al vuelo es despreciable frente al de decodificar la imagen.

IV. DIAGNÓSTICO DEL SESGO DE CONJUNTO

A. Observación inicial

El primer modelo entrenado, un ViT-B/16 con ajuste fino completo, alcanzó 82 % de exhaustividad sobre las imágenes sintéticas del conjunto de prueba. Al evaluarlo sobre 200 rostros generados en el momento con el mismo modelo StyleGAN3-R y semillas nuevas, la detección descendió a 23 %: peor que decidir al azar.

Dado que ambos conjuntos provenían nominalmente del mismo generador, la discrepancia solo podía explicarse por una diferencia sistemática entre las imágenes del conjunto público y las generadas directamente. Se plantearon tres hipótesis falsables.

B. Hipótesis 1: sesgo de compresión

Se comparó formato, modo de color, dimensiones, peso por píxel y tablas de cuantización JPEG sobre 300 imágenes de cada clase. Todas resultaron ser PNG sin pérdida en modo RGB a 256×256 . La diferencia en peso por píxel entre clases arrojó un tamaño de efecto de $d = 0,093$, despreciable. **Hipótesis descartada.**

C. Hipótesis 2: sesgo de remuestreo

Se observó que la energía en altas frecuencias de las imágenes generadas (6,806) quedaba fuera del rango de ambas clases del conjunto público (auténticas 6,673; sintéticas 6,606), con un tamaño de efecto $d \approx 0,80$ respecto a estas últimas. La hipótesis era que el filtro empleado al reducir de 1024 a 256 píxeles dejaba una firma distinta.

Se regeneraron las mismas 200 semillas aplicando seis filtros distintos (Lanczos, bicúbico, bilineal, área, vecino más próximo y reducción directa). Pese a que la energía en altas frecuencias varió entre 6,381 y 7,315, la tasa de detección permaneció entre 18,5 % y 21,5 %: una dispersión de tres puntos porcentuales.

Hipótesis descartada.

D. Hipótesis 3: sesgo de truncamiento

El parámetro ψ de truncamiento controla cuánto se aleja una muestra del centroide del espacio latente. Valores bajos producen rostros suavizados y próximos al promedio; valores altos, rostros diversos y texturados.

Se generaron 200 rostros para cada uno de cinco valores de ψ , evaluándolos con el mismo modelo. La tabla 2 muestra el resultado.

La relación es monótona y el rango abarca 77 puntos porcentuales. **Hipótesis confirmada.**

Para verificar que el generador era efectivamente el mismo, se compararon los perfiles de potencia radial obtenidos por transformada de Fourier bidimensional. La distancia L_2 normal-

TABLA 2
DETECCIÓN SEGÚN EL TRUNCAMIENTO DEL GENERADOR

ψ	Detección	$P(\text{sintética})$ media
0,40	100,0 %	0,9996
0,55	97,5 %	0,9678
0,70	76,5 %	0,7624
0,85	49,5 %	0,4919
1,00	23,0 %	0,2436

izada entre las sintéticas del conjunto público y las generadas resultó de 0,007 33, menor que la distancia entre las dos clases del propio conjunto (0,010 55). Mismo generador, distinta configuración.

E. Interpretación

El conjunto público fue generado con truncamiento bajo, de modo que todas sus muestras sintéticas comparten una suavidad característica. El clasificador aprendió a reconocer esa suavidad — una regla válida pero incompleta — en lugar de los artefactos del generador. Ante rostros diversos, que se aproximan estadísticamente a fotografías reales, la regla deja de aplicar.

Este tipo de sesgo es particularmente insidioso porque no es visible en ninguna métrica de validación convencional: el modelo obtiene resultados excelentes mientras se le evalúa sobre la misma distribución sesgada.

V. CORRECCIÓN Y RESULTADOS

A. Reconstrucción del conjunto

Se rediseñó el conjunto estratificando ψ para concentrar la mitad de las muestras sintéticas en el régimen difícil: 25 % en $[0,40, 0,70]$, 25 % en $[0,70, 0,85]$ y 50 % en $[0,85, 1,00]$, con media $\psi = 0,795$.

Se añadió augmentación aplicada por igual a ambas clases: espejo horizontal, compresión JPEG de calidad variable entre 70 y 100, ajustes de brillo, contraste y saturación, y desenfoque o realce. La compresión resultó la componente más relevante: al almacenarse los fragmentos como PNG sin pérdida, el modelo podía apoyarse en detalles de alta frecuencia que no sobreviven a ninguna imagen del mundo real.

El protocolo de evaluación reserva dos conjuntos generados con semillas disjuntas de las de entrenamiento (2 000 000–2 008 999): `val_hard` (semillas 2 500 000+) para seleccionar el punto de corte y detener el entrenamiento, y `test_hard` (semillas 2 600 000+) que no participa en ninguna decisión. Ambos con $\psi = 1,00$ y balance exacto de 500 imágenes por clase.

B. Evolución del sistema

La tabla 3 resume las cinco iteraciones sobre el conjunto de prueba difícil.

TABLA 3
EVOLUCIÓN SOBRE `TEST_HARD` ($\psi = 1,00$, SEMILLAS NUEVAS)

Versión	Aciertos	AUC	Detecta sint.
v1 — base	—	—	23,0 %
v2 — datos corregidos	—	—	60,9 %
v3 — ajuste de LR	89,8 %	0,9534	87,0 %
v4 — augmentación	92,0 %	0,9679	92,8 %
v5 — tres ViT	94,0 %	0,9832	96,2 %
ConvNeXt+Swin	99,7 %	0,9999	99,8 %

TABLA 4
DESEMPEÑO INDIVIDUAL POR ARQUITECTURA SOBRE `TEST_HARD`

Arquitectura	Parámetros	Aciertos	AUC
ViT-B/16	85,8 M	93,3 %	0,9778
Swin-T	27,5 M	97,3 %	0,9995
ConvNeXt-Tiny	27,8 M	99,6 %	0,9999

C. Sobre la resolución de entrada

Se consideró entrenar a 384 píxeles bajo la hipótesis de que los artefactos del generador residen en altas frecuencias. La idea fue descartada por un argumento de coherencia: las imágenes del conjunto público fueron distribuidas ya reducidas a 256×256 , de modo que ampliarlas no recupera detalle sino que lo interpola. Los rostros generados por nosotros, en cambio, parten de 1024×1024 reales. A 384 píxeles las sintéticas tendrían detalle genuino frente al detalle inventado de las auténticas, introduciendo exactamente el tipo de atajo que el trabajo buscaba eliminar.

Se optó por 256 píxeles, la resolución nativa del conjunto, evitando el descarte del 12 % de información que suponía reducir a 224. Al no ser 224 la resolución de preentrenamiento de las codificaciones posicionales del ViT, estas se interpolaron de una rejilla de 14×14 a una de 16×16 .

D. Comparación de arquitecturas

El resultado más inesperado del trabajo aparece en la tabla 4.

ConvNeXt-Tiny supera al Vision Transformer por más de seis puntos porcentuales empleando un tercio de los parámetros. La explicación propuesta remite al mecanismo de cada arquitectura: el ViT particiona la imagen en parches de 16×16 píxeles y proyecta cada uno a un vector, promediando implícitamente la información interna del parche. Los rastros que deja StyleGAN3 — textura de piel, transición del cabello, patrones del iris — operan a una escala menor que ese parche, de modo que se atenúan en la proyección. Las convoluciones jerárquicas, que recorren la imagen con filtros pequeños y construyen representaciones progresivamente más abstractas, preservan esa información.

TABLA 5
RENDIMIENTO DEL PREPROCESAMIENTO PARALELO

Procesos	Tiempo (s)	Imágenes/s
1	3,913	511,1
4	2,470	809,9
8	1,354	1477,1
12	1,061	1884,3
16	0,910	2198,3

E. Conjunto de modelos y punto de corte

Promediar las predicciones de ConvNeXt-Tiny y Swin-T, cada uno evaluado sobre la imagen y su reflejo horizontal, eleva el AUC a 0,9999. Resulta notable que este conjunto de dos modelos (55 M de parámetros) supere al de cinco que incluía los tres ViT (313 M), que alcanzaba 97,9 %.

El punto de corte se seleccionó sobre `val_hard` tomando el centro de la meseta de valores que quedan dentro del 0,4 % del óptimo, en lugar del máximo exacto. En iteraciones previas el máximo exacto oscilaba entre 0,05 y 0,94 de una época a otra — ruido, no señal — y llegó a producir un umbral que rendía peor que el valor por defecto. El criterio de meseta arrojó $\tau = 0,285$ sobre una región que abarca el 52 % de la rejilla explorada, indicando una separación entre clases lo bastante nítida como para que la elección exacta resulte poco crítica.

Aplicado a `test_hard`, este umbral logra 99,70 % frente al 99,90 % que habría obtenido el mejor umbral posible seleccionado sobre el propio conjunto de prueba. Esa diferencia de dos décimas es el costo metodológico de no inspeccionar el conjunto final, y se reporta explícitamente.

VI. ANÁLISIS DE RENDIMIENTO

A. Preprocesamiento paralelo

Se midió el rendimiento del preprocesamiento sobre 2 000 imágenes variando el número de procesos, con tres repeticiones por configuración y calentamiento previo de la caché del sistema de archivos. La tabla 5 recoge los resultados.

El rendimiento crece de forma monótona hasta un factor de 4,3 con dieciséis procesos. En la construcción del conjunto completo, la etapa de preprocesamiento sostuvo 2 396 imágenes/s sobre 9 000 imágenes.

1) Limitación de la medición

Debe señalarse una limitación metodológica. La referencia secuencial resultó inestable pese a tres estrategias correctivas sucesivas: calentamiento completo de la caché, fijación a un solo hilo de las bibliotecas numéricas mediante `OMP_NUM_THREADS` y variables equivalentes, y toma del mínimo entre repeticiones. La medición con dos procesos arrojó valores por debajo de la unidad, lo cual es físicamente implausible en una carga sin dependencias entre elementos.

La causa más probable es que el trabajo se ejecuta bajo un *cgroup* de SLURM que restringe a dieciséis de los veinte núcleos físicos del nodo, mientras el reparto interno entre el proceso padre y los hijos queda a criterio del planificador del sistema operativo. En consecuencia, el tiempo T_1 no corresponde a una ejecución estrictamente secuencial.

Por ese motivo se reporta el rendimiento absoluto, que es consistente entre corridas, y se omite el cálculo formal de *speedup* y eficiencia paralela, que requeriría un T_1 fiable. Una medición rigurosa exigiría un nodo en régimen exclusivo con afinidad de CPU fijada explícitamente.

B. Cuello de botella

El comportamiento observado sugiere que la carga está limitada por entrada/salida más que por cómputo. Las imágenes residen en un sistema de archivos Lustre compartido por red, y el rendimiento se aplanan al aproximarse a los doce procesos. La optimización natural consistiría en precargar el conjunto a almacenamiento local del nodo antes de procesar.

C. Entrenamiento en GPU

El entrenamiento de ConvNeXt-Tiny sostuvo 614 imágenes/s con precisión mixta, completando quince épocas sobre 16 800 imágenes en 7,9 min. Swin-T alcanzó 423 imágenes/s y diecisiete épocas en 13,1 min. La generación de rostros con StyleGAN3 sostuvo 36,8 imágenes/s a resolución 1024×1024 .

VII. REPRODUCIBILIDAD

Los rostros sintéticos no se almacenan: se registran sus semillas y el valor de ψ empleado, lo que permite regenerarlos de forma determinista sin coste de almacenamiento.

Esta propiedad se verificó de manera involuntaria pero concluyente. Una operación de limpieza de disco eliminó los modelos entrenados y el conjunto procesado. La reconstrucción completa — redescarga del conjunto público, regeneración de las 10 000 imágenes sintéticas y reentrenamiento de ambas arquitecturas — tomó treinta y cinco minutos y devolvió resultados idénticos hasta el cuarto decimal: AUC de 0,9999 y 0,9995 para ConvNeXt y Swin respectivamente, umbral $\tau = 0,285$, meseta de 95 valores. Se documenta aquí porque constituye una verificación de reproducibilidad más estricta que cualquier prueba planificada.

VIII. DISCUSIÓN Y LIMITACIONES

El sistema alcanza 99,7 % de aciertos, cifra que conviene interpretar con precisión. Lo que el modelo distingue son *rostros generados por StyleGAN3-R frente a fotografías del conjunto FFHQ*. No es un detector universal de contenido generado por inteligencia artificial: ante salidas de modelos de difusión como Stable Diffusion o Midjourney, o incluso de versiones anteriores de StyleGAN, cabe esperar un desempeño sustancialmente menor.

Esa expectativa se sustenta en el propio hallazgo del trabajo. Si un cambio en el parámetro de truncamiento del mismo generador bastó para reducir la detección del 82 % al 23 %, un cambio de arquitectura generativa producirá con seguridad un desplazamiento mayor. La lección es general: los detectores de contenido sintético aprenden la firma del generador con el que fueron entrenados, y la validación debe diseñarse para exponer esa dependencia en lugar de ocultarla.

Como línea de trabajo futura, la más pertinente sería evaluar la transferencia a otros generadores y, en su caso, entrenar sobre una mezcla de arquitecturas generativas para inducir invarianza.

IX. CONCLUSIONES

Un clasificador con métricas aparentemente satisfactorias sobre un conjunto público resultó estar explotando un artefacto de la construcción de ese conjunto. El diagnóstico requirió formular hipótesis falsables y descartarlas experimentalmente: dos resultaron incorrectas antes de identificar la causa real.

La corrección del sesgo elevó la detección del 23 % al 87 %, y el cambio de arquitectura la llevó al 99,8 %. Contra la expectativa inicial del proyecto — centrado en Vision Transformers desde su planteamiento — la arquitectura convolucional resultó marcadamente superior para esta tarea, con un tercio de los parámetros.

El trabajo deja dos conclusiones transferibles. La primera es que las métricas de validación no detectan sesgos compartidos entre los conjuntos de entrenamiento y prueba; validar contra datos generados bajo condiciones deliberadamente distintas es la única forma de exponerlos. La segunda es que la elección de arquitectura debe responder a la naturaleza de la señal buscada: cuando esa señal reside en texturas de escala fina, un transformador de parches grandes la atenúa antes de poder analizarla.

DISPONIBILIDAD

El código fuente, los cuadernos, los scripts de ejecución y los resultados experimentales están disponibles en <https://github.com/sklinderton/vit-real-vs-fake-faces>. Los conjuntos procesados y los modelos entrenados se excluyen del repositorio por su tamaño; el repositorio incluye los scripts y manifiestos de semillas que permiten regenerarlos íntegramente.

REFERENCIAS

- [1] I. J. Goodfellow *et al.*, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [2] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila, “Alias-free generative adversarial networks,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021.
- [3] A. Dosovitskiy *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations*, 2021.

- [4] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11976–11986.
- [5] Z. Liu *et al.*, “Swin Transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10012–10022.
- [6] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, “CNN-generated images are surprisingly easy to spot... for now,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 8695–8704.
- [7] Dask Development Team, “Dask: Library for dynamic task scheduling,” 2016. [En línea]. Disponible: <https://dask.org>
- [8] Polars Developers, “Polars: Fast multi-threaded DataFrame library,” 2023. [En línea]. Disponible: <https://pola.rs>